國立臺南大學資訊工程學系

資工三「演算法」課程
第 1 次作業

題目: Primality Testing

班級 ： 資工三

姓名 ： 呂建篁

學號 ： S10859045

老師：陳宗禧

中華民國 110 年 10 月 3 日

目錄

(一) 簡介及問題描述……3

(二) 理論分析……4

(三) 演算法則……5

(四) 程式設計環境架構……6

(五) 程式……7

(六) 執行結果、討論與心得……8

參考文獻資料……9~10

(一) 簡介及問題描述

1. 簡介

求 Prime(n)? 首先定義 一整數 n，判斷 n 是質數 or 合成數? …

2. 問題

Prime(n)問題定義：

求 Prime(n)，實作與分析

i. Basic Prime Testing
ii. Sieve of Eratosthenes
iii. Fermat Primality Testing
iv. Miller-Rabin Primality Testing

(二) 理論分析

1. Basic Prime Testing 理論

質數為 1 和自己本身相乘所組成的數，且找不到其餘因數，透過檢查小於等於$\sqrt{n}$ 的因數，則可不必檢查大於根號 n 的因數。

2. Sieve of Eratosthenes 理論

透過質數的倍數來判斷特定範圍內的合成數，若可以整除則必為合數，而通常最小的質數初始值固定從 2 開始測試。

3. Fermat Primality Testing 理論

依費馬小定理，若 p 為質數且$1 \leq a \leq p-1$，則$a^{p-1} \equiv 1(mod\ p)$，而 a 為隨機數，若等式成立，可能為質數也可能是合數，共以下三種可能的 case。

(i). n 為質數時， a^(n-1) % n = 1 必成立。

(ii). n 不為質數時，a^(n - 1) % n = 1 也可能成立。

(iii). 若以上述條件皆不成立，則不是質數。

4. Miller-Rabin Primality Testing 理論

Miller-Rabin 為 Basic Prime Testing + Fermat Primality Testing，而和其他三個質數判斷法最主要的差別就是將輸入的整數化為 $\boldsymbol{n-1=2^s*d}$ ，其中 d 必為奇數，依 Fermat 小定理，將 a^(n-1)取平方根後，若得出的結果為-1，則輸入的數必定是質數，反之得出的結果不是-1，則代表已經取盡 2 的冪次方。

(三) 演算法則

1. **Basic Prime Testing Algorithm)**

   time complexity: O($\sqrt{n}$)

   space complexity: O(1)

2. **Sieve of Eratosthenes Testing Algorithm**

   time complexity: O($nlogn$)

   space complexity: O($\sqrt{n}$)

3. **Fermat Primality Testing Algorithm**

   time complexity: O($klog^2n$)

   space complexity: O(1)

4. **Miller-Rabin Primality Testing Algorithm**

   time complexity: O($klog^2n$)

   space complexity: O(1)

(四) 程式設計環境架構

1. 程式語言

C++ in MS Windows

2. 程式開發工具

Visual C++ 2019

3. 電腦硬體

CPU: Intel Core i7 8550U

RAM: 8.00GB Single-Channel

4. 作業系統

Windows 10 家用版 64-bit

5. 顯示卡

Intel UHD Graphics 620

NVIDIA GeForce GTX 1050

(五) 程式 (含 source code, input code, and output code)

1. 主程式

1. 本次作業程式皆為 C++撰寫，原始碼檔案請參閱上船附檔。
2. 輸入輸出格式以下為可能顯示之範例，詳細 cout 結果得實際執行該程式。

2. Input Code Format

(1) 請輸入欲判斷的一整數: 2397

(2) 請輸入執行 loop 次數: 100

3. Output Code Format

(1) 您所輸入的 2397 不為質數

(2) 本次程式執行時間: 0.005 毫秒

(六) 執行結果、討論與心得

1. 執行結果

如(五)之 Output Code Format 所示

2. 討論 【執行時間效率】

依(三)演算法則之時間複雜度及空間複雜度推測，由左至右為執行所花費的時間長短。

Basic > Fermat Primality Testing = Miller-Rabin Primality Testing > Sieve of Eratosthenes

說明:

依(三)時間複雜度比較後，Basic 為最佳，Fermat 及 Miller 所需的時間複雜度兩者接近，而 Sieve 因用到 array 則會牽扯到空間複雜度，其時間複雜度劣於 Fermat 及 Miller，故將 Sieve 當作為最差。

3. 心得

藉由本次演算法作業撰寫，除了可以更加靈活運用大一所學之程式語言及大二資料結構外，藉由實務及理論並重的操作，能更加體會到好的程式除了擁有良好的程式風格外，更需要搭配資料結構及演算法，並將大問題切割成子問題逐一解決，才能使程式在執行上能更具效率性。

參考文獻資料

【Basic Prime Testing】

https://lakesd6531.pixnet.net/blog/post/254786875

https://stackoverflow.com/questions/4424374/determining-if-a-number-is-prime

https://www.csie.ntu.edu.tw/~b98902112/cpp_and_algo/cpp/practice_isprime_sol.html

https://zh.wikipedia.org/wiki/%E7%B4%A0%E6%80%A7%E6%B5%8B%E8%AF%95

http://www.trytoprogram.com/cpp-examples/cplusplus-program-to-check-prime-number/

【Sieve of Eratosthenes】

https://www.codespeedy.com/sieve-of-eratosthenes-in-cpp/

https://pjchender.blogspot.com/2017/09/algorithm-sieve-of-eratosthenes.html

https://gist.github.com/mcmullm2-dcu/117649ca592b8d6a065aa28db41b11dd

https://stackoverflow.com/questions/2582732/time-complexity-of-sieve-of-eratosthenes-algorithm

https://www.thelearningpoint.net/computer-science/c-program-using-the-sieve-of-eratosthenes-to-print-prime-numbers

https://cppsecrets.com/users/4895107101115104971181079798114974954641031099710510846991111109/Sieve-of-Eratosthenes.php

https://zh.wikipedia.org/wiki/%E5%9F%83%E6%8B%89%E6%89%98%E6%96%AF%E7%89%B9%E5%B0%BC%E7%AD%9B%E6%B3%95

【Fermat Primality Testing】

https://edisonx.pixnet.net/blog/post/61461627

https://en.wikipedia.org/wiki/Fermat_primality_test

https://cp-algorithms.com/algebra/primality_tests.html

https://stackoverflow.com/questions/15414970/fermat-primality-test

https://www.sanfoundry.com/c-program-perform-fermat-primality-test/

https://www.khanacademy.org/computing/computer-science/cryptography/random-algorithms-probability/v/fermat-primality-test-prime-adventure-part-10

【Miller-Rabin Primality Testing】

https://edisonx.pixnet.net/blog/post/61461627

https://www.itread01.com/content/1545750663.html

https://cp-algorithms.com/algebra/primality_tests.html

https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test

https://www.sanfoundry.com/cpp-program-implement-miller-rabin-primality-test/

https://zh.wikipedia.org/wiki/%E7%B1%B3%E5%8B%92%E6%8B%89%E5%AE%BE%E6%A3%80%E9%AA%8C

https://codereview.stackexchange.com/questions/189142/miller-rabin-primality-test-using-c-and-gmp